



Compte rendu

TP 03 : Traitement du Langage Naturel
TF-IDF et BERT

Réalisé par : Anas TARIK

Filière : 2CI-ISI

Année académique : 2025-2026

Encadrant : OMAR ZAHOUR

28 mai 2026

Table des matières

1	Introduction	2
2	Objectifs du TP	2
3	Environnement de travail	2
4	Partie I : TF-IDF	2
4.1	Principe de TF-IDF	2
4.2	Importation des bibliothèques	3
4.3	Préparation des données	3
4.4	Prétraitement	3
4.5	Application de TF-IDF	4
4.6	Analyse des mots les plus importants	4
4.7	Interprétation	5
5	Partie II : BERT	5
5.1	Présentation de BERT	5
5.2	Chargement du modèle BERT	5
5.3	Transformation des phrases en embeddings BERT	6
5.4	Interprétation des embeddings	7
5.5	Visualisation avec PCA	7
5.6	Interprétation	8
6	Partie III : Comparaison TF-IDF et BERT sur un dataset réel	8
6.1	Objectif	8
6.2	Chargement du dataset	8
6.3	Séparation des données	9
7	Classification avec TF-IDF	10
7.1	Code source	10
7.2	Interprétation	11
8	Classification avec BERT	11
8.1	Code source	11
8.2	Interprétation	13
9	Comparaison des résultats	13
9.1	Tableau comparatif	13
9.2	Visualisation graphique	13
9.3	Analyse comparative	14
10	Conclusion	14

1 Introduction

Le traitement automatique du langage naturel, ou NLP pour *Natural Language Processing*, est un domaine de l'intelligence artificielle qui permet aux machines d'analyser, comprendre et représenter le langage humain.

Dans ce TP, nous avons étudié deux approches importantes de représentation textuelle : TF-IDF et BERT. La première méthode, TF-IDF, est une approche statistique qui permet de mesurer l'importance d'un mot dans un document par rapport à un corpus. La deuxième méthode, BERT, est un modèle pré-entraîné basé sur les Transformers, capable de produire des représentations contextuelles des textes.

L'objectif général de ce TP est de comprendre comment transformer des documents textuels en représentations numériques, puis de les utiliser dans une tâche d'analyse ou de classification.

2 Objectifs du TP

Les objectifs de ce TP sont les suivants :

- comprendre le principe de TF-IDF ;
- préparer et nettoyer des données textuelles ;
- calculer les scores TF-IDF avec Scikit-learn ;
- interpréter les mots les plus importants dans chaque document ;
- comprendre le fonctionnement général de BERT ;
- transformer des phrases en embeddings avec BERT ;
- visualiser les embeddings avec PCA ;
- comparer TF-IDF et BERT sur une tâche de classification de sentiments.

3 Environnement de travail

Le TP a été réalisé avec l'environnement suivant :

- Python ;
- Jupyter Notebook ;
- NumPy et Pandas ;
- Scikit-learn ;
- NLTK ;
- PyTorch ;
- Hugging Face Transformers ;
- Matplotlib et Seaborn.

4 Partie I : TF-IDF

4.1 Principe de TF-IDF

TF-IDF signifie *Term Frequency-Inverse Document Frequency*. Cette méthode permet d'évaluer l'importance d'un mot dans un document par rapport à l'ensemble du corpus.

Elle repose sur deux idées principales :

- un mot fréquent dans un document peut être important pour ce document ;
- un mot présent dans beaucoup de documents est moins discriminant.

Ainsi, TF-IDF donne un poids élevé aux mots qui sont fréquents dans un document mais rares dans les autres documents.

4.2 Importation des bibliothèques

```
1 import numpy as np
2 import pandas as pd
3 import string
4
5 from sklearn.feature_extraction.text import TfidfVectorizer
```

4.3 Préparation des données

Nous avons utilisé trois documents simples :

```
1 documents = [
2     "I love machine learning.",
3     "BERT is an amazing NLP model.",
4     "We are studying embeddings."
5 ]
6
7 print("Documents originaux :")
8 for i, doc in enumerate(documents, 1):
9     print(f"Document {i} :", doc)
```

4.4 Prétraitement

Le prétraitement consiste à supprimer la ponctuation et à convertir le texte en minuscules.

```
1 def preprocess(doc):
2     return doc.translate(str.maketrans('', '', string.punctuation))
3         .lower()
4
5 documents_clean = [preprocess(doc) for doc in documents]
6
7 print("Documents après traitement :")
8 for i, doc in enumerate(documents_clean, 1):
9     print(f"Document {i} :", doc)
```

```

Documents originaux :
Document 1 : I love machine learning.
Document 2 : BERT is an amazing NLP model.
Document 3 : We are studying embeddings.

... Documents après prétraitement :
Document 1 : i love machine learning
Document 2 : bert is an amazing nlp model
Document 3 : we are studying embeddings

```

FIGURE 1 – Documents originaux et documents après prétraitement

4.5 Application de TF-IDF

Nous avons ensuite appliqué la vectorisation TF-IDF avec `TfidfVectorizer`.

```

1 vectorizer = TfidfVectorizer(stop_words='english')
2
3 tfidf_matrix = vectorizer.fit_transform(documents_clean)
4
5 feature_names = vectorizer.get_feature_names_out()
6
7 df_tfidf = pd.DataFrame(
8     tfidf_matrix.toarray(),
9     columns=feature_names
10 )
11
12 df_tfidf

```

```

...
   amazing  bert  embeddings  learning   love  machine  model  nlp  studying
0        0.0   0.0   0.000000   0.57735  0.57735  0.57735   0.0   0.0   0.000000
1        0.5   0.5   0.000000   0.00000  0.00000  0.00000   0.5   0.5   0.000000
2        0.0   0.0   0.707107   0.00000  0.00000  0.00000   0.0   0.0   0.707107

```

FIGURE 2 – Matrice TF-IDF obtenue

4.6 Analyse des mots les plus importants

Pour chaque document, nous avons récupéré le mot ayant le score TF-IDF le plus élevé.

```

1 max_value_indices = np.argmax(df_tfidf.values, axis=1)
2
3 max_words = [feature_names[idx] for idx in max_value_indices]

```

```
4
5 for doc_num, word in enumerate(max_words, 1):
6     print(f"Document {doc_num}: Mot le plus important -> {word}")
```

```
• Document 1: Mot le plus important -> learning
  Document 2: Mot le plus important -> amazing
  Document 3: Mot le plus important -> embeddings
```

FIGURE 3 – Mot le plus important pour chaque document

4.7 Interprétation

Les résultats montrent que TF-IDF permet de repérer les mots les plus représentatifs de chaque document. Par exemple, le mot `learning` est important dans le premier document, tandis que `amazing` ressort dans le deuxième document.

Cette méthode est simple, rapide et efficace pour représenter les textes sous forme numérique. Cependant, elle ne prend pas en compte le contexte profond des mots.

5 Partie II : BERT

5.1 Présentation de BERT

BERT signifie *Bidirectional Encoder Representations from Transformers*. Il s'agit d'un modèle pré-entraîné développé pour comprendre le contexte des mots dans les deux directions : de gauche à droite et de droite à gauche.

Contrairement à TF-IDF, BERT ne représente pas uniquement les mots selon leur fréquence. Il produit des embeddings contextuels, c'est-à-dire des vecteurs qui dépendent du sens du mot dans la phrase.

5.2 Chargement du modèle BERT

```
1 import torch
2 from transformers import BertTokenizer, BertModel
3
4 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
5 model = BertModel.from_pretrained("bert-base-uncased")
6
7 print("Tokenizer chargé :", tokenizer.__class__.__name__)
8 print("Modèle chargé :", model.__class__.__name__)
9 print("BERT est prêt à être utilisé.")
```

```
vocab.txt: 232k/? [00:00<00:00, 4.43MB/s]
tokenizer.json: 466k/? [00:00<00:00, 11.2MB/s]
config.json: 100% [570/570 [00:00<00:00, 20.1kB/s]]
model.safetensors: 100% [440M/440M [00:04<00:00, 143MB/s]]
Loading weights: 100% [199/199 [00:00<00:00, 276.82it/s, Materializing param=pooler.dense.weight]]
BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |

Notes:
- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Tokenizer chargé : BertTokenizer
Modèle chargé : BertModel
BERT est prêt à être utilisé.
```

FIGURE 4 – Chargement du tokenizer et du modèle BERT

Le message affiché lors du chargement peut contenir des avertissements liés à certaines couches non utilisées. Ces avertissements peuvent être ignorés dans notre cas, car nous utilisons BERT comme extracteur d’embeddings.

5.3 Transformation des phrases en embeddings BERT

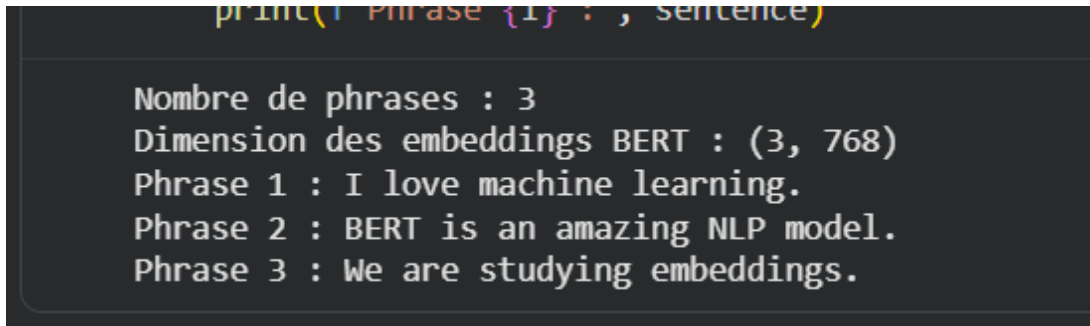
Nous avons utilisé les mêmes phrases que dans la partie TF-IDF.

```
1 sentences = [
2     "I love machine learning.",
3     "BERT is an amazing NLP model.",
4     "We are studying embeddings."
5 ]
6
7 def sentence_to_embedding(sentence, model, tokenizer):
8     inputs = tokenizer(
9         sentence,
10        return_tensors="pt",
11        truncation=True,
12        padding=True,
13        max_length=512
14    )
15
16    with torch.no_grad():
17        outputs = model(**inputs)
18
19    return outputs["last_hidden_state"][:, 0, :].numpy()
20
21 embeddings = [
22     sentence_to_embedding(sentence, model, tokenizer)
23     for sentence in sentences
24 ]
25
26 embeddings = np.array(embeddings).squeeze()
```

```

27
28 print("Nombre de phrases :", len(sentences))
29 print("Dimension des embeddings BERT :", embeddings.shape)
30
31 for i, sentence in enumerate(sentences, 1):
32     print(f"Phrase {i} :", sentence)

```



```

print(f"Phrase {i} :", sentence)

Nombre de phrases : 3
Dimension des embeddings BERT : (3, 768)
Phrase 1 : I love machine learning.
Phrase 2 : BERT is an amazing NLP model.
Phrase 3 : We are studying embeddings.

```

FIGURE 5 – Dimensions des embeddings générés par BERT

5.4 Interprétation des embeddings

Le résultat obtenu est de la forme :

(3, 768)

Cela signifie que nous avons 3 phrases, et que chaque phrase est représentée par un vecteur de 768 dimensions. Ces vecteurs contiennent des informations contextuelles sur le sens des phrases.

5.5 Visualisation avec PCA

Comme les embeddings BERT ont 768 dimensions, il est impossible de les visualiser directement. Nous avons donc utilisé la méthode PCA afin de réduire la dimensionnalité à 2 dimensions.

```

1 from sklearn.decomposition import PCA
2 import matplotlib.pyplot as plt
3
4 pca = PCA(n_components=2)
5 embeddings_2d = pca.fit_transform(embeddings)
6
7 plt.figure(figsize=(9, 6))
8 plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1])
9
10 for i, sentence in enumerate(sentences):
11     plt.annotate(
12         sentence,
13         (embeddings_2d[i, 0], embeddings_2d[i, 1]),
14         fontsize=10
15     )
16

```



```
17 plt.title("Visualisation des embeddings BERT avec PCA")
18 plt.xlabel("Composante principale 1")
19 plt.ylabel("Composante principale 2")
20 plt.grid(True)
21 plt.show()
```

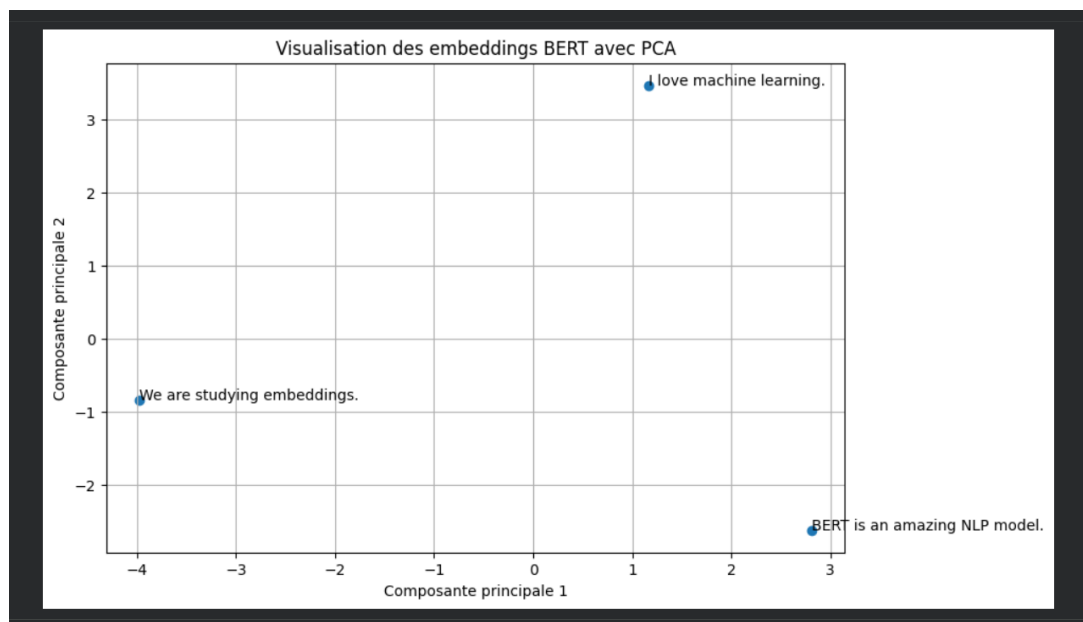


FIGURE 6 – Visualisation des embeddings BERT avec PCA

5.6 Interprétation

Chaque point du graphique représente une phrase. La position des points est obtenue après réduction des embeddings BERT de 768 dimensions vers 2 dimensions.

Si deux phrases sont proches dans cet espace réduit, cela signifie que leurs représentations vectorielles sont relativement similaires. Cependant, il faut noter qu'une partie de l'information est perdue pendant la réduction de dimension.

6 Partie III : Comparaison TF-IDF et BERT sur un dataset réel

6.1 Objectif

Dans cette partie, nous avons comparé TF-IDF et BERT sur une tâche de classification de sentiments. Le dataset utilisé est `movie_reviews` de la bibliothèque NLTK. Il contient des critiques de films positives et négatives.

L'objectif est de transformer les textes avec deux méthodes différentes, puis d'entraîner un modèle de régression logistique pour comparer les performances.

6.2 Chargement du dataset

```

1 import nltk
2 from nltk.corpus import movie_reviews
3 from sklearn.model_selection import train_test_split
4
5 nltk.download("movie_reviews")
6
7 documents_dataset = [
8     (list(movie_reviews.words(fileid)), category)
9     for category in movie_reviews.categories()
10    for fileid in movie_reviews.fileids(category)
11 ]
12
13 sentences = [" ".join(words) for words, label in documents_dataset]
14
15 labels = [1 if label == "pos" else 0 for words, label in
16           documents_dataset]
17
18 print("Nombre total de documents :", len(sentences))
19 print("Nombre total de labels :", len(labels))
20 print("Exemple de texte :")
21 print(sentences[0][:500])
22 print("\nLabel du premier texte :", labels[0])

```

```

... [nltk data] Downloading package movie_reviews to /root/nltk_data...
[nltk data] Unzipping corpora/movie_reviews.zip.
Nombre total de documents : 2000
Nombre total de labels : 2000
Exemple de texte :
plot : two teen couples go to a church party , drink and then drive . they get into an accident . one of the guys dies , but his girlfriend continues to see him in her life
Label du premier texte : 0

```

FIGURE 7 – Chargement du dataset Movie Reviews

6.3 Séparation des données

Les données ont été séparées en deux parties :

- 80% pour l'entraînement ;
- 20% pour le test.

```

1 sentences_train, sentences_test, y_train, y_test = train_test_split
2     (
3         sentences,
4         labels,
5         test_size=0.2,
6         random_state=42
7     )
8
9 print("Nombre de textes d'entraînement :", len(sentences_train))
10 print("Nombre de textes de test :", len(sentences_test))
11 print("Exemple train :", sentences_train[0][:300])
12 print("Label train :", y_train[0])

```

```

Nombre de textes d'entraînement : 1600
Nombre de textes de test : 400
Exemple train : while watching loser , it occurred to me that amy heckerling ' s true genius as a film - maker is casting . in fast times at ridgemont high , she gave us sean
Label train : 0

```

FIGURE 8 – Séparation des données en entraînement et test

7 Classification avec TF-IDF

7.1 Code source

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.metrics import classification_report, accuracy_score
4
5 vectorizer = TfidfVectorizer(
6     stop_words="english",
7     max_features=5000
8 )
9
10 X_train_tfidf = vectorizer.fit_transform(sentences_train)
11 X_test_tfidf = vectorizer.transform(sentences_test)
12
13 clf_tfidf = LogisticRegression(max_iter=1000)
14 clf_tfidf.fit(X_train_tfidf, y_train)
15
16 y_pred_tfidf = clf_tfidf.predict(X_test_tfidf)
17
18 acc_tfidf = accuracy_score(y_test, y_pred_tfidf)
19
20 print("Résultats avec TF-IDF")
21 print("Accuracy TF-IDF :", round(acc_tfidf * 100, 2), "%")
22 print()
23 print(classification_report(y_test, y_pred_tfidf))

```

Résultats avec TF-IDF				
Accuracy TF-IDF : 82.25 %				
	precision	recall	f1-score	support
0	0.82	0.82	0.82	199
1	0.82	0.83	0.82	201
accuracy			0.82	400
macro avg	0.82	0.82	0.82	400
weighted avg	0.82	0.82	0.82	400

FIGURE 9 – Résultats de classification avec TF-IDF

7.2 Interprétation

Le modèle TF-IDF associé à la régression logistique donne une bonne performance pour la classification des critiques de films. Cette méthode est rapide et efficace, surtout lorsque les données ne nécessitent pas une compréhension contextuelle très profonde.

8 Classification avec BERT

8.1 Code source

Pour éviter un temps d'exécution trop long, nous avons utilisé un sous-ensemble du dataset : 300 textes pour l'entraînement et 100 textes pour le test.

```
1 import numpy as np
2 import torch
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import classification_report, accuracy_score
5
6 N_TRAIN = 300
7 N_TEST = 100
8
9 sentences_train_small = sentences_train[:N_TRAIN]
10 sentences_test_small = sentences_test[:N_TEST]
11 y_train_small = y_train[:N_TRAIN]
12 y_test_small = y_test[:N_TEST]
13
14 def sentence_to_embedding(sentence, model, tokenizer):
15     inputs = tokenizer(
16         sentence,
17         return_tensors="pt",
18         truncation=True,
19         padding=True,
20         max_length=512
21     )
22
23     with torch.no_grad():
24         outputs = model(**inputs)
25
26     return outputs["last_hidden_state"][:, 0, :].numpy()
27
28 X_train_bert = []
29
30 for i, sentence in enumerate(sentences_train_small):
31     emb = sentence_to_embedding(sentence, model, tokenizer)
32     X_train_bert.append(emb)
33
34     if (i + 1) % 50 == 0:
35         print(f"Embeddings train g n r s : {i + 1}/{N_TRAIN}")
36
37 X_test_bert = []
38
```

```

39 for i, sentence in enumerate(sentences_test_small):
40     emb = sentence_to_embedding(sentence, model, tokenizer)
41     X_test_bert.append(emb)
42
43     if (i + 1) % 20 == 0:
44         print(f"Embeddings test g n r s : {i + 1}/{N_TEST}")
45
46 X_train_bert = np.concatenate(X_train_bert, axis=0)
47 X_test_bert = np.concatenate(X_test_bert, axis=0)
48
49 print("Shape X_train_bert :", X_train_bert.shape)
50 print("Shape X_test_bert :", X_test_bert.shape)
51
52 clf_bert = LogisticRegression(max_iter=1000)
53 clf_bert.fit(X_train_bert, y_train_small)
54
55 y_pred_bert = clf_bert.predict(X_test_bert)
56
57 acc_bert = accuracy_score(y_test_small, y_pred_bert)
58
59 print("\nR sultats avec BERT")
60 print("Accuracy BERT :", round(acc_bert * 100, 2), "%")
61 print()
62 print(classification_report(y_test_small, y_pred_bert))

```

```

Embeddings train générés : 50/300
Embeddings train générés : 100/300
Embeddings train générés : 150/300
Embeddings train générés : 200/300
Embeddings train générés : 250/300
Embeddings train générés : 300/300
Embeddings test générés : 20/100
Embeddings test générés : 40/100
Embeddings test générés : 60/100
Embeddings test générés : 80/100
Embeddings test générés : 100/100
Shape X_train_bert : (300, 768)
Shape X_test_bert : (100, 768)

Résultats avec BERT
Accuracy BERT : 74.0 %

```

	precision	recall	f1-score	support
0	0.72	0.72	0.72	47
1	0.75	0.75	0.75	53
accuracy			0.74	100
macro avg	0.74	0.74	0.74	100
weighted avg	0.74	0.74	0.74	100

FIGURE 10 – Résultats de classification avec les embeddings BERT

8.2 Interprétation

Avec BERT, chaque critique est transformée en embedding contextuel. Ces embeddings sont ensuite utilisés comme entrée pour un modèle de régression logistique.

BERT est plus riche que TF-IDF car il tient compte du contexte des mots. Cependant, il demande plus de ressources et plus de temps de calcul. Dans notre expérience, nous avons limité le nombre d'exemples pour accélérer l'exécution.

9 Comparaison des résultats

9.1 Tableau comparatif

```

1 comparison = pd.DataFrame({
2     "M thode": [
3         "TF-IDF + Logistic Regression",
4         "BERT embeddings + Logistic Regression"
5     ],
6     "Accuracy": [acc_tfidf, acc_bert]
7 })
8
9 comparison["Accuracy (%)"] = comparison["Accuracy"] * 100
10
11 print("Comparaison des performances :")
12 comparison

```

Comparaison des performances :

	Méthode	Accuracy	Accuracy (%)
0	TF-IDF + Logistic Regression	0.8225	82.25
1	BERT embeddings + Logistic Regression	0.7400	74.00

FIGURE 11 – Tableau de comparaison entre TF-IDF et BERT

9.2 Visualisation graphique

```

1 plt.figure(figsize=(8, 5))
2 plt.bar(comparison["M thode"], comparison["Accuracy (%)"])
3 plt.ylabel("Accuracy (%)")
4 plt.title("Comparaison des performances : TF-IDF vs BERT")
5 plt.xticks(rotation=20, ha="right")
6 plt.ylim(0, 100)
7 plt.grid(axis="y")
8 plt.show()

```

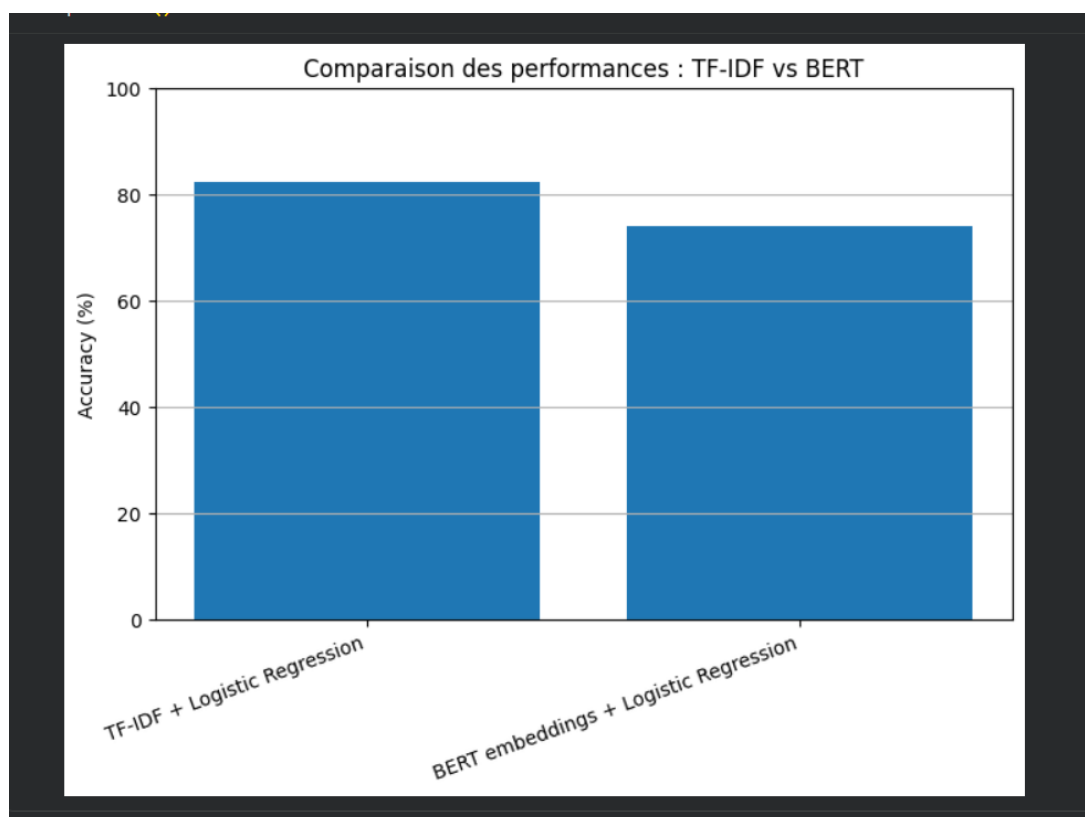


FIGURE 12 – Visualisation graphique des performances TF-IDF et BERT

9.3 Analyse comparative

TF-IDF est une méthode rapide, simple et efficace. Elle fonctionne bien pour des tâches classiques de classification de texte, surtout lorsque les mots importants suffisent à distinguer les classes.

BERT est une méthode plus avancée, car elle produit des embeddings contextuels. Il peut mieux comprendre le sens des phrases, mais son coût computationnel est plus élevé. Dans notre cas, BERT a été appliqué sur un sous-ensemble réduit du dataset pour limiter le temps d'exécution.

Le choix entre TF-IDF et BERT dépend donc du besoin :

- si l'objectif est d'avoir une solution rapide et légère, TF-IDF est un bon choix ;
- si l'objectif est d'obtenir une représentation contextuelle plus riche, BERT est plus adapté ;
- si les ressources matérielles sont limitées, TF-IDF est plus pratique ;
- si le sens profond du texte est important, BERT peut donner de meilleurs résultats.

10 Conclusion

Ce TP nous a permis d'étudier deux approches différentes pour représenter le texte sous forme numérique.

Dans la première partie, nous avons utilisé TF-IDF afin de calculer l'importance des mots dans un corpus. Cette méthode est simple, rapide et permet d'obtenir des représentations efficaces pour la classification.

Dans la deuxième partie, nous avons utilisé BERT pour transformer les phrases en embeddings contextuels. Chaque phrase est représentée par un vecteur de 768 dimensions, ce qui permet de capturer des informations plus riches que TF-IDF.

Enfin, dans la troisième partie, nous avons comparé TF-IDF et BERT sur une tâche de classification de sentiments. Cette comparaison montre que TF-IDF est rapide et performant, tandis que BERT est plus puissant mais plus coûteux en calcul.

En conclusion, TF-IDF reste une méthode très utile pour des tâches simples ou rapides, alors que BERT est plus adapté aux applications nécessitant une meilleure compréhension du contexte.